Report

Robot Localization

Dinh-Son Vu

Table of Contents

1. Pre-requisites 3
2. Tasks 3
3. Explanations 5
4. Packages, nodes, and topics 6
5. Future works 7

1. Pre-requisites

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is simulation only. The hardware is not required.

2. Tasks

The localization of a robot is one of the hardest challenges in mobile robots. The encoder of the robot gives the pose (position and orientation) of a robot, but it can drift due to slippage of the wheel on the floor, inaccuracy of the encoders, dimension error of the robot geometry, etc. An IMU is a valuable addition to the system, as it is not affected by slippage. However, the IMU is sensitive to noise and vibration, thus an average of both sensors would be necessary: this is the purpose of the robot_localization package and its Kalman filter.

Tasks	Code
Install localization **Install transform tools**	sudo apt install ros-jazzy-robot-localization sudo apt install ros-jazzy-tf2-tools
Clone the repo from github	git clone https://github.com/ROS2-rmitbot/lesson3_ws.git
Navigate to the workspace **Remove the folders build install log** **Build the packages** **Source the workspace** **Open VSCode**	cd ~/lesson3_ws rm -rf build install log colcon build source install/setup.bash code .
Launch rviz, gazebo, controller, teleopkeyboard	ros2 launch rmitbot_description display.launch.py ros2 launch rmitbot_description gazebo.launch.py ros2 launch rmitbot_controller controller.launch.py ros2 launch rmitbot_controller teleopkeyboard.launch.py
Launch local localization	ros2 launch rmitbot_localization local_localization.launch.py
Alternatively, launch the bringup launch	ros2 launch rmitbot_bringup rmitbot.launch.py
Visualize the tf tree	ros2 run tf2_tools view_frames evince <frame.pdf>

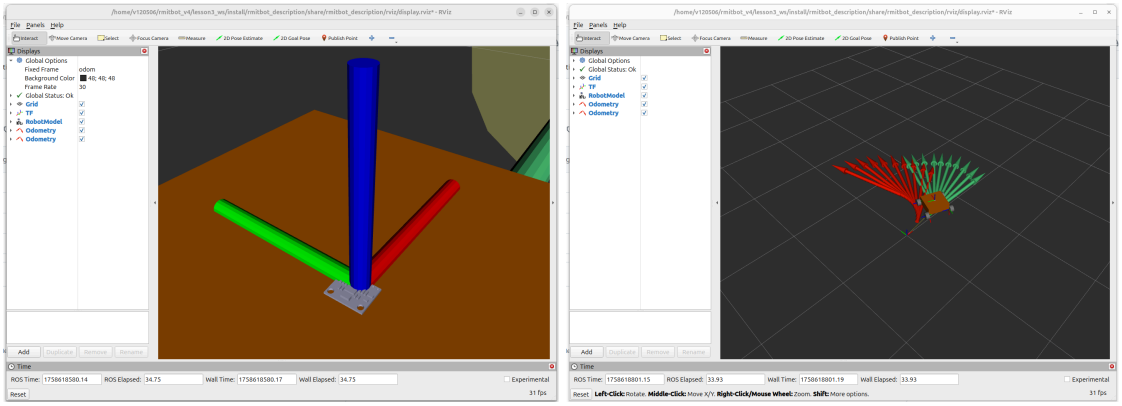

Figure 1: The IMU is located on top of the robot. Its addition is beneficial as the robot slips quite often during rotation.

There are two important frames, which are odom and base_footprint. They respectively correspond to the initial pose of the robot and its current pose. Initially, the transform (tf) between odom and base_footprint is broadcast by the ros2_control package (obtained by the encoders). The localization package outputs a better estimate since it uses the fusion between imu and encoders. Thus, the parameter “enable_odom_tf” in the controller yaml file must be set to false.

3. Explanations

The pose estimate can come from different sensors, namely:

- encoders: also called odometry or dead reckoning.
- imu: inertial measurement unit
- gps (outdoor): absolute positioning of the robot
- lidar odometry: LiDAR odometry estimates motion by aligning successive LiDAR scans
- visual odometry: works better with a depth camera and is similar to lidar odometry.
- Visual apriltag: tag are used to give absolute positioning of the robot

Tasks done in each package:

- rmitbot_description:
    - Add a new IMU in the robot_description urdf.xacro (loading stl file graphics)
    - Add the stl file in the meshes folder (already inside)
    - Add gazebo.xacro information (add plugin imu, add noise for more realistic physics)
    - Add gz_ros2_bridge for the imu (clock@rosgraph already added to use sim_time)
    - Add dependencies in the package.xml file
- rmitbot_localization: new package
    - ekf.yaml: defines the parameters for the extended Kalman filter (core of the robot_localization)
    - local_localization.launch.py: launch file to start the local localization

Package modification

rmitbot_description:

- meshes/imu_link.STL added
- urdf/rmitbot_imu..xacro added
- launch/gazebo.launch.py: gz_ros2_bridge added (required for imu plugin)
- package.xml: updated to include ros_gz_bridge

rmitbot_localization:

- ekf.yaml: defines the parameters for the extended Kalman filter

4. Packages, nodes, and topics

Packages	Nodes	Description
rmitbot_bringup		Launch the other packages and nodes
rmitbot_description	display.launch.py: rviz, rsp gazebo.launch.py: gazebo	Launch the urdf and static tf
rmitbot_controller	Controller manager: jsb spawner, controller spawner	Launch input/output of the robot
rmitbot_localization	ekf	Launch the sensor fusion between wheel encoder and imu

Package	Topic	Description
ROS2 standard	/parameter_events /rosout	n/a n/a
rviz	/clicked_point /goal_pose /initialpose	Click point in rviz Target pose of the robot (for nav2) Initial pose of the robot (for slam_toolbox)
gazebo	/clock	Sim time for node synchronization
robot_state_publisher	/robot_description /tf /tf_static	Description of the robot in xml format Dynamic tf (odom --> base_footprint) Static tf (base_footprint --> rest of the robot)
controller_manager	/controller_manager/activity /controller_manager/introspection_data/full /controller_manager/introspection_data/names /controller_manager/introspection_data/values	For debugging only
ros2_controllers	/joint_states /rmitbot_controller/cmd_vel /rmitbot_controller/odom /rmitbot_controller/cmd_vel_out /joint_state_broadcaster/transition_event /dynamic_joint_states /rmitbot_controller/transition_event	Important: pos, vel, effort of the joint Important: Cartesian vel command Important: odometry (pose of the robot)
robot_localization	/imu/out /odometry/filtered	Imu data (actually from gazebo, but used in robot_localization) Odom from ekf: filtered and less prone to drift than /rmitbot_controller/odom

5. Future works

- The robot will drift, lose some encoder counts, and slip on some surface. Can you quantify the error in orientation with plotjuggler?